



Programmiertechnik II

Ungerichtete Graphen

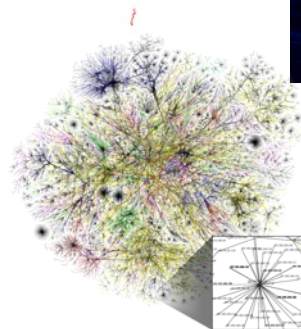
Ralf Herbrich

1. Begriffe
2. Repräsentationen
3. Tiefensuche
 - Zusammenhangskomponenten
4. Breitensuche
 - Kevin-Bacon Zahl

1. **Begriffe**
2. Repräsentationen
3. Tiefensuche
 - Zusammenhangskomponenten
4. Breitensuche
 - Kevin-Bacon Zahl

Ungerichtete Graphen

- **Definition (Graph).** Ein **Graph** $G = (V, E)$ besteht aus einer Menge V von Knoten (*vertices, nodes*) und einer Menge $E \subseteq V \times V$ von Kanten.
- **Definition (Ungerichteter Graph).** Ein Graph $G = (V, E)$ ist ungerichtet falls $\forall (v, v') \in E \rightarrow (v', v) \in E$.
- **Graphen in der echten Welt:**
 - Karten & Transportwege
 - Soziale Netzwerke
 - Computernetzwerke
- **Probleme** in echten Graphen:
 1. (Kurze) Pfade finden (*path finding*)
 2. Cluster finden (*connected components*)
 3. Verbundenheit (*connectivity*)



Programmiertechnik II

Unit 9a – Ungerichtete Graphen

Graphen in der echten Welt (*ctd*)

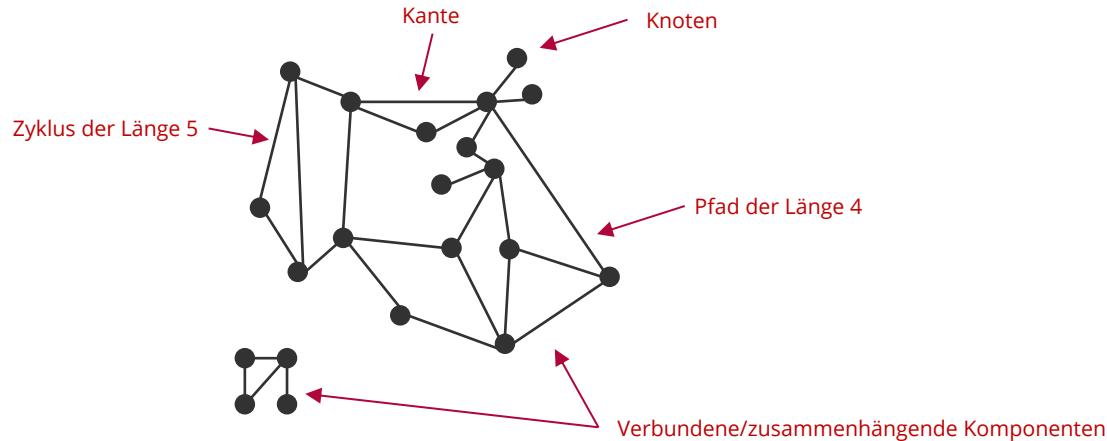
- Graphen sind eine perfekte Abstraktion für Interaktionen von physikalischen Entitäten und tauchen ständig im Alltag auf.

Graph	Knoten	Kanten
Kommunikation	Telephone, Computer	Optisches Kabel
Transport	Kreuzung	Straße
Soziales Netzwerk	Person	Freundschaft
Neuronaes Netzwerk	Neuron	Synapse
Molekül	Atom	Verbindung
Schaltkreise	Prozessor, Register	Leitung
Finanzen	Aktie/Währung	Kauf/Verkauf
Brettspiel	Stellung	Legalzer Zug

Programmiertechnik II

Unit 9a – Ungerichtete
Graphen

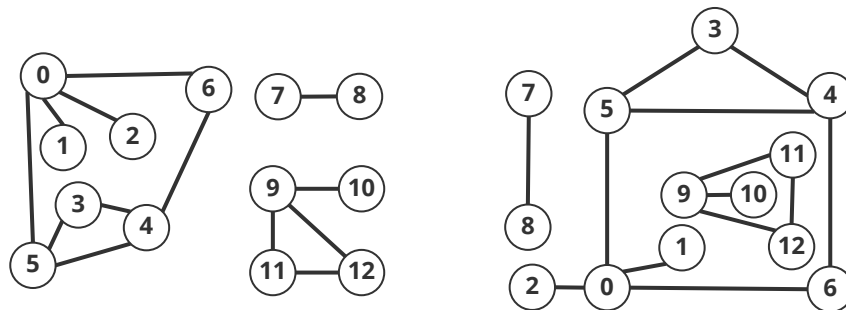
- **Definition (Pfad).** Eine Folge von Kanten $e_1, e_2, \dots, e_n$ heißt **Pfad der Länge n** genau dann wenn für alle i : $e_i = (v', v)$, $e_{i+1} = (v, v'')$ und $v' \neq v''$.
- **Definition (Zyklus).** Ein Pfad $e_1, e_2, \dots, e_n$ heißt **zyklisch** genau dann wenn $e_{1,1} = e_{n,2}$ (das heißt, der erste und letzte Knoten im Pfad sind identisch).
- **Definition (Verbundenheit).** Zwei Knoten u und v sind **verbunden** wenn ein Pfad $e_1, e_2, \dots, e_n$ existiert so dass $e_1 = (u, v')$ und $e_n = (v'', v)$.



1. Begriffe
2. **Repräsentationen**
3. Tiefensuche
 - Zusammenhangskomponenten
4. Breitensuche
 - Kevin-Bacon Zahl

Mögliche Repräsentationen

- **Grafische Repräsentation:** Gibt eine Intuition über die Struktur des Graphs
 - Hängt sehr von der Position der Knoten ab!
- **Adjazenzmatrix Repräsentation:** Benutze eine Matrix adj mit $|V| \times |V|$ `bool` Einträgen
 - Für jede Kante zwischen Knoten v und w wird adj wie folgt verändert:
 $adj[v][w] = adj[w][v] = \text{true}$
 - Für "dünne" Graphen ($|E| = O(|V|)$), brauchen wir trotzdem $O(|V|^2)$ Speicher



	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	1	1	0	0	1	1	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	1	1	0	0	0	0	0	0	0
4	0	0	0	1	0	1	1	0	0	0	0	0	0
5	1	0	0	1	1	0	0	0	0	0	0	0	0
6	1	0	0	0	1	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	1	0	0	0	0
8	0	0	0	0	0	0	0	0	1	0	0	0	0
9	0	0	0	0	0	0	0	0	0	1	1	1	1
10	0	0	0	0	0	0	0	0	0	1	0	0	0
11	0	0	0	0	0	0	0	0	0	1	0	0	1
12	0	0	0	0	0	0	0	0	0	1	0	1	0

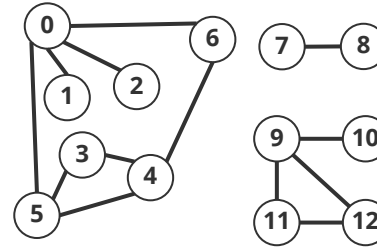
Zwei Einträge pro Kante

Programmiertechnik II

Unit 9a – Ungerichtete Graphen

Adjazenzlistenrepräsentation

- **Idee:** Für jeden Knoten wird eine verlinkte Liste der Knoten gespeichert, zu denen der Knoten verbunden ist
- In der Praxis benutzt, weil
 1. Algorithmen oft über die Nachbarknoten iterieren
 2. Echte Graphen "dünn" (*sparse*) sind

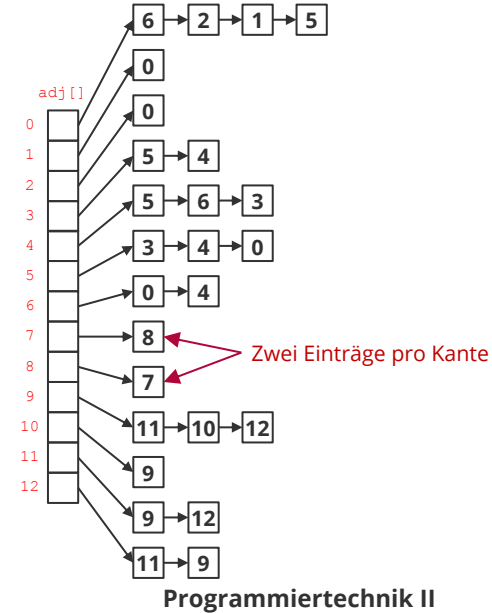


```

// Implements a class representing an undirected graph
class Graph {
    int no_vertices; // number of vertices
    int no_edges; // number of edges
    Bag<int*> adj_lists; // array of adjacency lists

public:
    // constructor
    Graph(const int V) : no_vertices(V), no_edges(0) {
        adj_lists = new Bag<int*>[V];
    }

    // adds an edge to the undirected graph
    void add_edge(const int, const int) {
        no_edges++;
        adj_lists[v].add(w);
        adj_lists[w].add(v);
        return;
    }
};
    
```



Repräsentation	Speicher	Kante hinzufügen	Kanten (v, w) vorhanden?	Iterieren Nachbarn (v)
Kantenliste	$O(E)$	$O(1)$	$O(E)$	$O(E)$
Adjazenzmatrix	$O(V ^2)$	$O(1)$	$O(1)$	$O(V)$
Adjazenzliste	$O(E + V)$	$O(1)$	$O(\text{degree}(v))$	$O(\text{degree}(v))$

Programmiertechnik II

Unit 9a – Ungerichtete Graphen

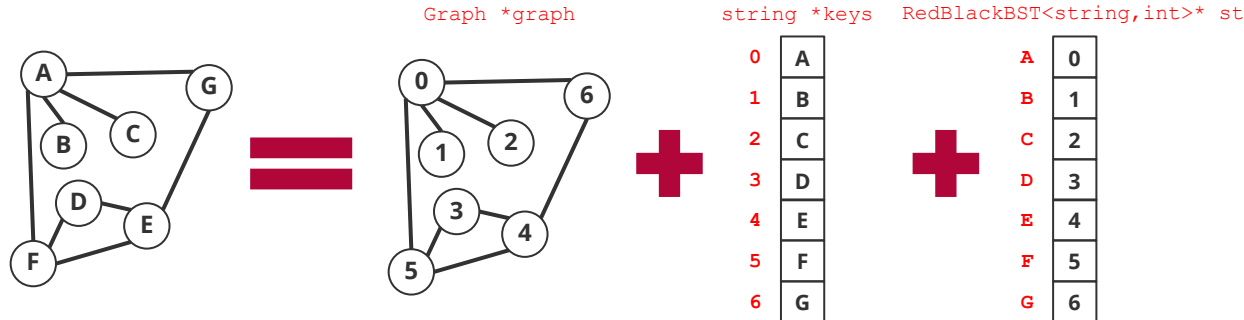
graph.h

9/25

Graphen mit Knotennamen

- **Problem:** Wie stellen wir Graphen da, bei denen die Knoten Zeichenketten als Namen haben?
 - Der Typ `Graph` geht davon aus, dass alle Knoten von $0, \dots, |V| - 1$ nummeriert sind
- **Lösung:** Wir benutzen Symboltabellen, um eindeutige Abbildungen zwischen den Zahlen $0, \dots, |V| - 1$ und den Knotennamen zu speichern!

[symbol_graph.h](#)



```
class SymbolGraph {
    RedBlackBST<string, int>* st; // string -> index
    string* keys; // index -> string
    Graph* graph; // the underlying graph

public:
    // constructor
    SymbolGraph(const string&, const char);

    // returns whether or not the graph contain the vertex named s
    bool contains(const string& s) const { return (st->contains(s)); }

    // returns the integer associated with the vertex named s
    int index_of(const string& s) const { return (st->get(s)); }

    // returns the name of the vertex associated with the integer
    string name_of(const int v) const { return (keys[v]); }

    // returns the associated graph
    const Graph* get_graph() const { return (graph); }
};
```

Programmiertechnik II

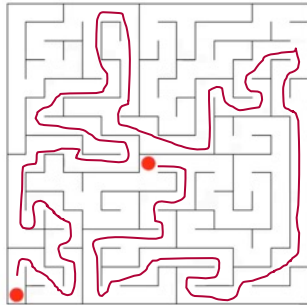
Unit 9a – Ungerichtete
Graphen

1. Begriffe
2. Repräsentationen
- 3. Tiefensuche**
 - Zusammenhangskomponenten
4. Breitensuche
 - Kevin-Bacon Zahl

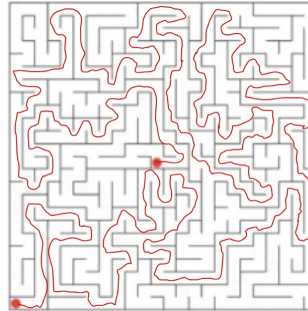
Aufwärmbeispiel

■ **Problem:** Weg in einem Labyrinth finden

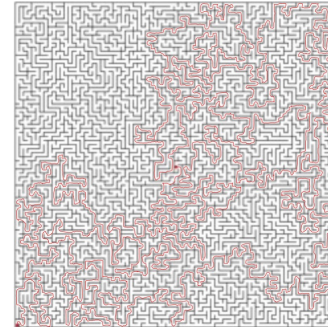
Leicht



Mittel



Schwer



Tempel des Theseus
(Paphos, Zypern)

■ **Theseus's Algorithmus**

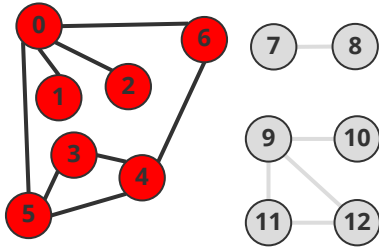
1. **Beginne** an Startpunkt im Labyrinth und gehe den rechtesten Weg mit einem Faden auf dem Boden
2. **Markiere** jede Verzweigung
3. **Wenn** man an einer Verzweigung ankommt, die schon markiert ist, geh den Faden zurück bis zur letzten Verzweigung, die noch einen Weg hat, der nicht besucht wurde



Programmiertechnik II

Unit 9a – Ungerichtete
Graphen

- **Ziel:** Systematisch alle Knoten u von einem Startknoten v aus besuchen
- **Idee:** Theseus's Labyrinthexploration nachahmen
 - `marked` markiert schon besuchte Knoten
 - `edge_to[w] == v` bedeutet, dass die Kante nach w von v kommt



v	marked	edge_to
0	T	-
1	T	0
2	T	0
3	T	5
4	T	6
5	T	4
6	T	0
7	F	-
8	F	-
9	F	-
10	F	-
11	F	-
12	F	-

```
// depth first search from v
void dfs(const Graph& g, const int v) {
    marked[v] = true;
    for (auto w : g.adj(v)) {
        if (!marked[w]) {
            dfs(g, w);
            edge_to[w] = v;
        }
    }
}
```

[dfs.h](#)

Programmiertechnik II

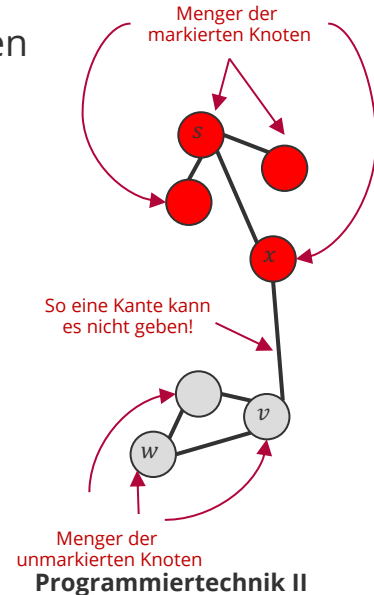
Unit 9a – Ungerichtete
Graphen

Eigenschaften der Tiefensuche

- **Satz:** Die Tiefensuche markiert alle Knoten, die mit einem gegebenen Startknoten $s \in V$ verbunden sind, in einer Zeit proportional zur Summe ihrer Grade.
- **Beweis:** Alle Knoten die mit dem Startknoten verbunden sind, werden markiert
 1. Jeder markierte Knoten ist mit s verbunden, da der Algorithmus nur Kanten folgt.
 2. Alle verbundenen Knoten sind markiert (Widerspruchsbeweis):
 - Nehmen wir an, ein unmarkierter Knoten w ist mit s verbunden.
 - Dann gibt es eine Kante auf jedem Pfad von s zu w , welche alle markierten und alle unmarkierten Knoten verbindet (sagen wir x zu v).
 - Das ist aber ein Widerspruch, da der Algorithmus v markiert hätte.

Der Algorithmus prüft Markiertheit in einer Zeitkomplexität proportional zum Grad/Knoten.

- **Satz:** Die Tiefensuche berechnet einen Pfad von einem gegebenen Startknoten $s \in V$ zu jedem verbundenen Knoten $w \in V$ in einer Zeit die proportional zur Länge des Pfades ist.
- **Beweis:** Das Feld `edge_to` repräsentiert einen Baum mit der Wurzel s und den Kindknoten aller verbundenen Knoten.



Unit 9a – Ungerichtete Graphen

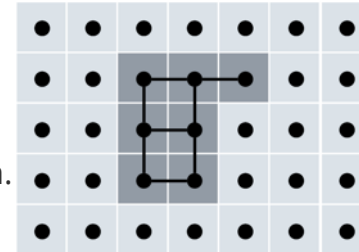
Anwendung der Tiefensuche

- **Problem:** Zusammenhängende Flächen auf digitalen Fotos füllen



- **Lösung:** (Impliziter) Aufbau eines Rastergraphen

- **Knoten:** Pixel
- **Kante:** Zwischen zwei benachbarten Pixeln, die eine ähnliche Intensität haben
- **Fläche:** Die zusammenhängende Komponente des Rastergraphen.



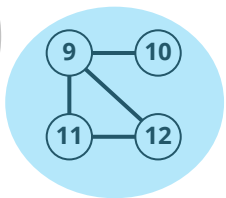
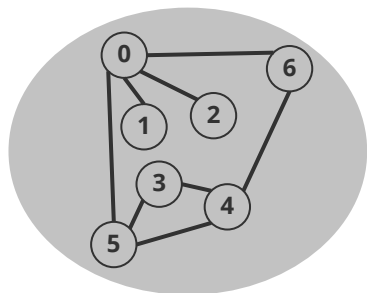
Programmiertechnik II

Unit 9a – Ungerichtete
Graphen

1. Begriffe
2. Repräsentationen
3. Tiefensuche
 - **Zusammenhangskomponenten**
4. Breitensuche
 - Kevin-Bacon Zahl

Zusammenhangskomponente

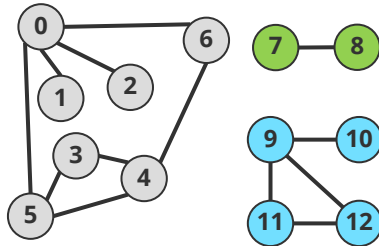
- Die Relation $\text{connected}(v, w)$ (v ist zusammenhängend mit w) ist eine **Äquivalenzrelation**:
 - Reflexivität**: $\text{connected}(v, v) \in R$
 - Symmetrie**: $\text{connected}(v, w) \in R \Rightarrow \text{connected}(w, v) \in R$
 - Transitivität**: $\text{connected}(u, v) \in R \wedge \text{connected}(v, w) \in R \Rightarrow \text{connected}(u, w) \in R$
- Definition (Zusammenhangskomponente)**: Eine Zusammenhangskomponente eines Graphen $G = (V, E)$ ist eine maximale Menge verbundener Knoten.



v	cc_id
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	1
8	1
9	2
10	2
11	2
12	2

Zusammenhangskomponenten und Tiefensuche

- **Ziel:** Jedem Knoten v einen Index der Zusammenhangskomponente zuweisen
- **Idee:** Tiefensuche von jedem potentiellen Startknoten aus benutzen, um eine Zusammenhangskomponente zu finden
 - `marked` markiert schon besuchte Knoten
 - `cc_id[v]` ist der Index der Zusammenhangskomponente von v



v	marked	cc_id
0	T	0
1	T	0
2	T	0
3	T	0
4	T	0
5	T	0
6	T	0
7	T	1
8	T	1
9	T	2
10	T	2
11	T	2
12	T	2

```
// Implements a class determining the connected components in an undirected graph
class CC {
    int no_vertices; // total number of vertices
    bool* marked; // marked[v] = is there an s-v path?
    int* cc_id; // cc_id[v] = cc_id of connected component containing v
    int no_components; // number of connected components

    // depth first search from v
    void dfs(const Graph& g, const int v) {
        marked[v] = true;
        cc_id[v] = no_components;
        for (auto w : g.adj(v)) {
            if (!marked[w]) dfs(g, w);
        }
    }

public:
    // constructor
    CC(const Graph& g) : no_components(0), no_vertices(g.V()) {
        // initialize the marked, cc_id and cc_size array
        marked = new bool[no_vertices];
        cc_id = new int[no_vertices];
        for (auto i = 0; i < no_vertices; i++) {
            marked[i] = false;
            cc_id[i] = -1;
        }

        // run depth-first search recursively
        for (auto v = 0; v < no_vertices; v++) {
            if (!marked[v]) {
                dfs(g, v);
                no_components++;
            }
        }
    }
}
```

[cc.h](#)

Programmiertechnik II

Unit 9a – Ungerichtete
Graphen

1. Begriffe
2. Repräsentationen
3. Tiefensuche
 - Zusammenhangskomponenten
4. **Breitensuche**
 - Kevin-Bacon Zahl

Motivation: Breitensuche

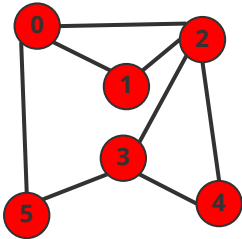
- **Tiefensuche:** Berechnet, **ob** es einen Pfad zwischen zwei Knoten $s \in V$ und $w \in V$ gibt und ermittelt ihn effizient.
- **Kürzeste-Pfade Problem:** Wir möchten für einen gegebenen Graphen und einen Startknoten $s \in V$ (effizient) den **kürzesten** Pfad von s zu einem gegebenen Zielknoten $v \in V$ berechnen.
 - Breitensuche erlaubt dies, indem wir nicht in die “Tiefe” (mit Hilfe von Rekursion/Stack) sondern in the “Breite” (mit Hilfe von Warteschlange) gehen
 - Tiefen- und Breitensuche unterscheiden sich nur durch die Containerstruktur!
- **Breitensuche Algorithmus**
 1. **Füge** den Startpunkt in eine Warteschlange und markiere ihn
 2. Solange die Warteschlange **nicht leer ist**
 - **Entferne** den Kopf der Warteschlange
 - Überprüfe alle Nachbarn, ob sie schon markiert sind; wenn nicht, markiere den Nachbarn und füge den Nachbarknoten zur Warteschlange hinzu



Programmiertechnik II

Unit 9a – Ungerichtete
Graphen

- **Ziel:** Kürzeste Pfade von einem gegebenen Startknoten berechnen
- **Idee:** Alle Pfade der gleichen Länge hintereinander in einer Warteschlange
 - `marked` markiert schon besuchte Knoten
 - `edge_to[w] == v` bedeutet, das die Kante nach `w` von `v` kommt
 - `dist_to` speichert, wie lang der kürzeste Pfad zum Startknoten ist



v	marked	edge_to	dist_to
0	T	-	0
1	T	0	1
2	T	0	1
3	T	2	2
4	T	2	2
5	T	0	1

```
// breadth first search from a single source s
void bfs(const Graph& g, const int s) {
    Queue<int> q;

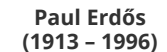
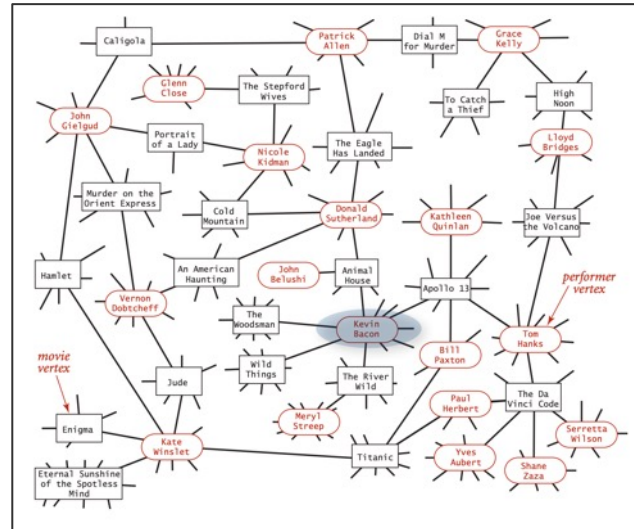
    // initialize distances and visitation
    for (auto v = 0; v < no_vertices; v++) {
        dist_to[v] = std::numeric_limits<int>::max();
        edge_to[v] = -1;
        marked[v] = false;
    }
    dist_to[s] = 0;
    marked[s] = true;
    q.enqueue(s);

    // run BFS by processing the queue
    while (!q.is_empty()) {
        int v = q.dequeue();
        for (auto w : g.adj(v))
            if (!marked[w]) {
                edge_to[w] = v;
                dist_to[w] = dist_to[v] + 1;
                marked[w] = true;
                q.enqueue(w);
            }
    }

    return;
}
```

1. Begriffe
2. Repräsentationen
3. Tiefensuche
 - Zusammenhangskomponenten
4. Breitensuche
 - **Kevin-Bacon Zahl**

- ## degree of separation.cpp



Unit 9a – Ungerichtete Graphen

- Breiten- und Tiefensuche erlauben effiziente Lösungen für viele (aber nicht alle!) Graphprobleme

Problem	Breitensuche	Tiefensuche	Zeit
Pfad zwischen s und v ?	✓	✓	$O(V + E)$
Kürzester Pfad zwischen s und v ?	✓		$O(V + E)$
Zusammenhangskomponenten	✓	✓	$O(V + E)$
Zyklus finden	✓	✓	$O(V + E)$
Euler-Zyklus		✓	$O(V + E)$
Bipartitheit	✓	✓	$O(V + E)$
Planarität		✓	$O(V + E)$

Viel Spaß bis zur nächsten Vorlesung!